

Final Project

Total points: 150

Due Date: 04/30/2026

Building Machine Learning Models for Wearable Physiological Signal Classification

In this project, you will perform exploratory data analysis (EDA), apply dimensionality reduction using PCA, and build machine learning models using **Python** to classify physiological states captured by wearable devices. You will explore several machine learning techniques and evaluate their effectiveness on the dataset. You can use existing packages/libraries to implement the assignment, but make sure to **thoroughly comment on the code** clearly explaining what you did. Follow the instructions in JupyterNotebook_updated installation guide on pilot for setting up the environment. **Analysis and Interpretation** are crucial aspects of the assignment, so try to answer the question in more detail to receive full credit. Create a PDF format report answering the questions in detail and attach any graphs/charts/visualizations you may have. Don't directly copy paste cell outputs, create tables for confusion matrices, classification reports etc. Don't just write code, comment your code wherever necessary, to help navigate.

The dataset (extracted_features_3label.csv) contains physiological signals recorded from wearable sensors worn by participants performing yoga postures. The five sensor modalities are: accelerometer (body motion), blood volume pulse (BVP, cardiovascular activity), electrodermal activity (EDA, sweat response), skin temperature, and RR interval (heart rhythm). For each modality, eight statistical features are extracted per 60-second window, giving roughly 40 numerical features in total. The dataset also includes categorical metadata: gender, posture type, yoga experience, difficulty level, and subject identifier. The target variable state has three classes: relaxed, parasympathetic, and sympathetic. You are responsible for encoding all categorical variables into numeric form before modeling.

1. Exploratory Data Analysis [30 points]

Start by getting a solid understanding of the dataset before building any models. Good EDA will inform your decisions throughout the rest of the project.

- Load the dataset, explore its dimensions and handle any missing values. List all categorical columns (e.g., gender, posture_type, yoga_experience, difficulty_level, state) and convert them to numeric values using a suitable encoding method. [5]
- Examine the class distribution of the target variable (state) and visualize it using appropriate charts (e.g., bar plots). [5]
- Rather than one large heatmap for all 40+ features, compute and visualize the correlation matrix separately for each sensor modality — accelerometer, BVP, EDA, temperature, and RR interval (5 heatmaps total). Within each group, identify which features are most correlated with each other and flag any potentially redundant pairs. Does this separation help with the analysis? [10]
- For each sensor modality, compute descriptive statistics (e.g., mean and standard deviation) of its features across the three target classes. Based on these, identify which modality exhibits the greatest variation between classes. [5]
- Create box plots for one feature from each sensor modality (e.g. heart rate, EDA, accelerometer), grouped by the three target classes (relaxed, parasympathetic, sympathetic). Do any features show clear separation between classes? Discuss your observations. [5]

2. Principal Component Analysis [30 points]

With roughly 40 features — many of which are correlated within the same sensor modality — directly feeding all of them into a model can introduce noise and redundancy. PCA addresses this by creating a smaller set of new features called principal components, where each component is a weighted combination of the original features. The first component captures the most variance in the data, the second captures the next most, and so on. You choose how many components to keep based on how much total variance they explain together.

- Separate your features (X) from the target variable (y). Standardize X using StandardScaler — PCA is sensitive to differences in scale across features. Then apply PCA across all components and produce two plots: (i) a bar chart of the explained variance ratio per component, and (ii) a cumulative explained variance curve with a horizontal reference line at 90%. Report the shape of the scaled feature matrix before transformation. [10]
- Based on your scree plot, choose the minimum number of components needed to retain at least 90% of the total variance. Justify your choice. Apply PCA with this number of components: use fit_transform on the training set and transform on the test set. Print the shape of the dataset before PCA and after PCA side by side. Additionally, create a scatter plot of the first two principal components colored by the target variable. Comment on whether there is any visible separation between classes. This reduced dataset (X_train_pca, X_test_pca) will be your input for all remaining parts. [10]
- Compute the factor loadings for your selected components (loadings = eigenvectors $\times$ sqrt(explained variance), as shown in the class notebook). For each of the first three principal components, present the top five contributing original features in a table sorted by absolute loading value. Which sensor modalities appear most frequently among the top contributors? Does this connect to the high-correlation patterns you identified in Part 1e? [10]

For implementing machine learning models, it is necessary to separate features from the predictor variable and split data into training and test sets (sometimes validation set as well). Here, for the assignment, split the data into train and test sets. Load the required model, train it and predict the output variable. Evaluate model performance using metrics such as F1-score, accuracy, precision, recall etc. This is the backbone of a machine learning model implementation. Based on the distribution of data and other constraints, several other components can be added to help deal with them. For example, we employ stratification and cross-validation to handle class imbalance and feature scaling to normalize the data etc. Likewise, regularization helps in handling overfitting of the models. **Use the PCA reduced dataset from Part 2 as input for training the models in Parts 3, 4 and 5.**

3. Logistic Regression [25 points]

- a. Implement Logistic Regression model on the PCA-reduced dataset without any regularization. Evaluate the model performance. [5]
- b. Implement logistic regression with L1 (Lasso) regularization and evaluate model performance [5]
- c. Implement logistic regression with L2 (Ridge) regularization and evaluate model performance [5]
- d. Discuss the impact of regularization on model performance and what difference did you notice from 3a, 3b and 3c? [8]
- e. Also state any assumptions you made about the data split. Consider: if the same subject appears in both the training and test sets, could this affect how well results generalize to new subjects? How would you address this? [2]

To implement neural networks, feel free to use pytorch or tensorflow libraries.

- Use 'Adam' as optimizer while training the models
- Experiment with different values of learning rate, batch size, number of neurons per layer, activation functions, and dropout rates

4. Neural Networks [25 points]

- a. Design and test two different neural network architectures, one being a simple feedforward neural network (e.g., 1-2 hidden layers) and the other being a deeper neural network (e.g., 3+ hidden layers, dropout for regularization) [15]
- b. Compare the performance of these architectures and provide a summary of the architectures, training process, and evaluation metrics (accuracy, loss, precision, recall, F1-score, confusion matrix etc.) [10]

5. Test with Different Cross-Validation Folds [20 points]

- a. Implement k-fold cross-validation on Logistic regression with different values of k (e.g., 5, 10) and provide your observations [10]
- b. Test cross-validation with and without stratification and provide your observations. Analyze the impact of the number of folds and stratification on model performance [10]

SHAP (SHapley Additive exPlanations) is a tool for explaining what your model has learned — which features are driving predictions and in which direction. Apply SHAP to your best-performing model from Part 3 or Part 4.

6. SHAP analysis [20 points]

- a. Create a SHAP Explainer for your chosen model and compute SHAP values on the test set. Generate (i) a global feature importance bar plot and (ii) a beeswarm plot. Which principal components have the greatest overall influence on predictions? Do any components push predictions consistently in one direction? Do the components identified as most important by SHAP align with the sensor modalities that showed the greatest class variation in Part 1d? Discuss whether the model's learned importance agrees with your earlier statistical observations. [10]
- b. Select a single test sample and generate a SHAP waterfall plot to explain that individual prediction. Describe which components pushed the prediction toward or away from a particular class and what this might suggest about the underlying physiological signals. [5]
- c. Select another single test sample and repeat the operation from b. [5]

7. Bonus Question [5 points]

- a. If you had access to the full raw sensor streams from this study, what would you explore next — a different task, a different modeling approach, or a question this dataset could not answer? There is no right answer here.

References

- Sample Logistic Regression implementation <https://www.geeksforgeeks.org/ml-logistic-regression-using-python/>
- Sample Neural network implementation <https://www.geeksforgeeks.org/feedforward-neural-network/>
- Sample PCA implementation <https://www.geeksforgeeks.org/principal-component-analysis-with-python/>
- PCA visualization <https://www.kaggle.com/code/prashantkumarsundge/understanding-pca-in-details>
- Scikit-learn Cross Validation https://scikit-learn.org/stable/modules/cross_validation.html
- SHAP Documentation: <https://shap.readthedocs.io/en/latest/>

Submission Instructions

You can use NumPy, pandas, Matplotlib, scikit-learn, pytorch, tensorflow etc required libraries for your assignment, make sure to understand it conceptually and answer the questions with proper analysis

- Please upload a zipped file named 'FinalProject_YourName' to Dropbox. The zip file should contain the following items: a dataset, code file(s) (preferably .ipynb format) (please use relative paths when reading/importing the dataset), a PDF-format report, and a README.txt.
- Please ensure that your code is error-free; there should be no errors or warnings when running it on my machine.
- If you are using a programming language other than Python, please provide a README.txt file explaining how to run your code to obtain the result. Mention any libraries or dependencies that need to be installed before running the code, if necessary.

Academic Integrity

Discussion of course contents with other students is an important part of the academic process and is encouraged. However, it is expected that course programming assignments, homework assignments, and other course assignments will be completed on an individual basis (unless specified otherwise). Students may discuss general concepts with one another, but may not, under any circumstances, work together on the actual implementation of any course assignment. If you work with other students on “general concepts” be certain to acknowledge the collaboration and its extent in the assignment. Unacknowledged collaboration will be considered dishonest. “Code sharing” (including code from previous quarters) is strictly disallowed. “Copying” or significant collaboration on any graded assignments will be considered a violation of the university guidelines for academic honesty.

If the same work is turned in by two or more students, all parties involved will be held equally accountable for violation of academic integrity. You are responsible for ensuring that other students do not have access to your work: do not give another student access to your account, do not leave printouts in the recycling bin, pick up your printouts promptly, do not leave your workstation unattended, etc. If you suspect that your work has been compromised notify me immediately. If you have any questions about collaboration or any other issues related to academic integrity, please contact me immediately for clarification. In addition to the policy stated in this syllabus, students are expected to comply with the Wright State University Code of Student Conduct

(<http://www.wright.edu/students/judicial/conduct.html>) and the portions Pertaining to Academic Integrity (<http://www.wright.edu/students/judicial/integrity.html>) at all times. Note: In cases where there is suspicion of academic dishonesty, the professor and teaching assistant reserve the right to address the matter by calling in the student for an in-person question and answer session.

